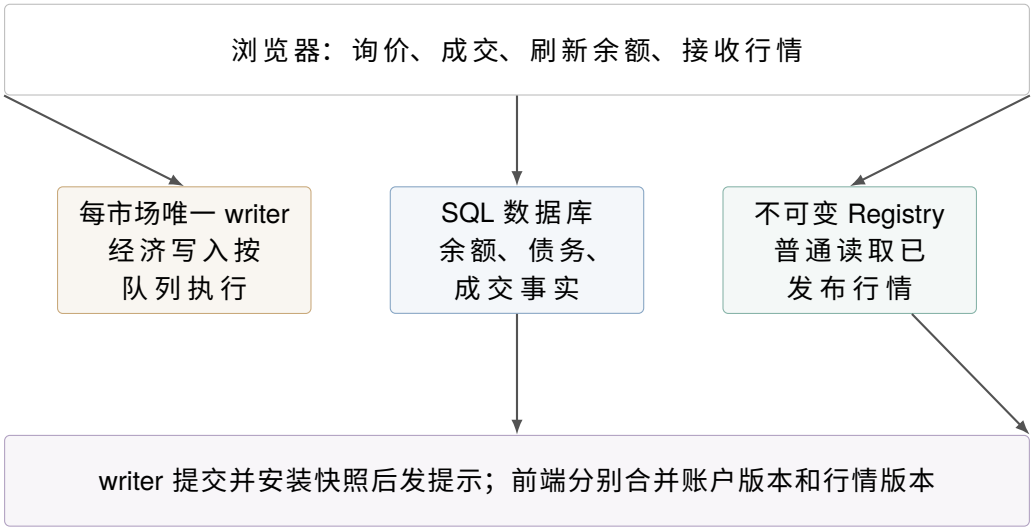
交易请求的一生

唯一 writer、账户锁与已提交行情

点击报价、提交成交、刷新余额，服务端走的是不同路径。本指南沿着请求往前走，说明谁负责排队、谁保存资金事实，以及读者什么时候能看到新行情。

> **阅读基准**。本文描述 2026-10-18 已确认的迁移目标，替代检查点 `330f4e6` 中的通用 GATE 协调。实施、审阅和最终性能对照的进度，以仓库迁移计划及其 SDD ledger 为准；本文不代表迁移已全部完成、验证通过或已经部署。



图例：蓝色为 SQL；**绿色为内存**；**棕色为排队与锁**；**紫色为通知**。时序图从上向下看时间，图中长度不代表真实耗时。

想弄清什么	从这里读起
数据库、writer 和快照各管什么	第2页，数据与锁的分工
报价、成交和融资的顺序	第3–5页，FX 请求
同账户跨市场如何避免重复授信	第5页，User 行锁与新账本
账户、公共 SSE、私人 SSE 的承诺	第7–9页

1 先认清数据、队列和锁

FX 与 LMSR 共用已提交行情的读取方式，执行状态仍有区别。FX 首轮保留事务内数据库权威；LMSR 沿用原有 writer 内存份额推进交易，再把结果持久化。

数据	SQL 中是什么	内存中是什么
用户资金	现金、债务、钱包、持仓、利息与账户版本	不新增余额或债务缓存;执行检查必须重读本人账本
FX 当前市场	储备、费率、状态与 `pool_version`；成交计算仍使用事务内真实值	每 pair 一个 writer 排序；Registry 保存完整不可变的已提交快照
LMSR 当前市场	参数与 Outcome 份额的持久镜像	writer 保存可推进的 q; Registry 保存不可变读取投影
成交和执行结果	成交记录、回执、幂等键、订单终态与审计	价格快照不替代这些持久证据
历史与订阅	K 线、历史版本、游标等持久记录	现有历史环、runtime、订阅队列与共享广播；各有自己的覆盖协议

四种等待，作用各不相同

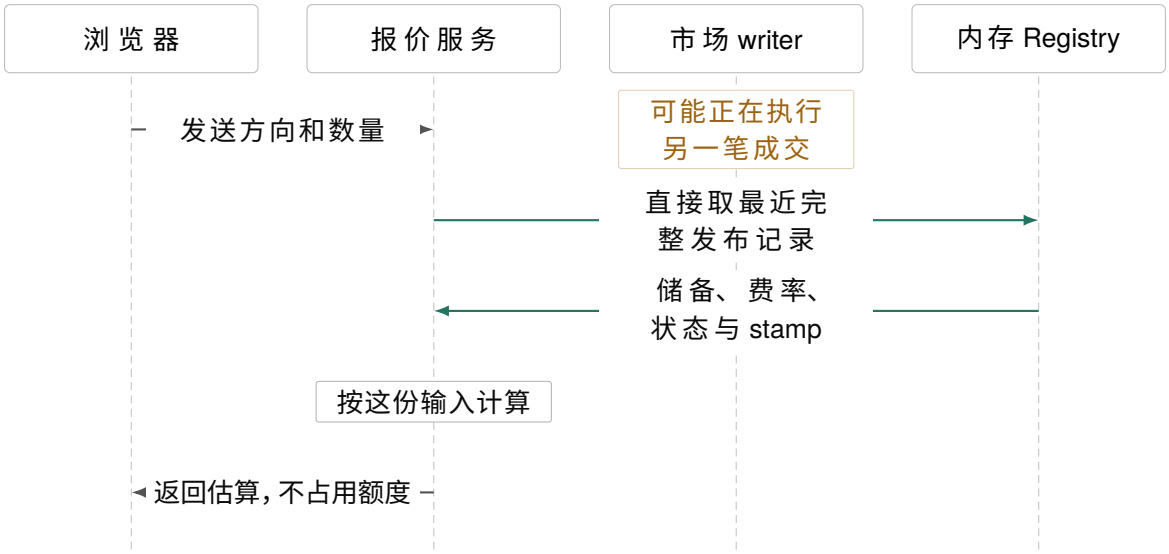
1. **市场命令队列**：同市场经济写入按成功入队顺序执行；不同市场可以同时运行。
2. **User 行锁**：同账户即使进入不同市场队列，也要串行检查和修改本人资金。取得锁后重读，提交或回滚后释放。
3. **其他数据库行锁**：继续保护 Pair、钱包、持仓、订单等；精确公共历史读取仍可能使用 `FOR SHARE`。
4. **连接池**：分配数据库连接。它会影响延迟，不负责市场或账户的业务排序。

> 通用 GATE 的职责被拆回明确边界：市场 writer 管写入次序，数据库事务和行锁管资金，Registry 供普通读使用。写实例 OWNERSHIP 继续保留，进程内队列不能代替跨实例所有权。

代码入口：`services/fx/writer.py`; `services/market_writer.py`; `services/market_read_state.py`

2 报价：读取一份完整的已提交行情

纯 FX 报价用储备和手续费估算兑换结果，不借款、不扣余额、不写成交。LMSR quote 同样从 Registry 取得定价输入；静态名称等元数据仍可读库。



不等待正在写的市场。writer 正在执行或队列里还有命令时，普通读直接取最后一份完整快照。数据库已提交而内存尚未安装的短暂窗口，也允许返回上一份快照；响应的 stamp 和行情 `as_of` 描述实际附带的记录。

冷读没有发布权限。活跃 owner 市场先完成 bootstrap，才公开可接单状态。它缺少健康快照时，普通活跃行情返回暂不可用，经济写入由 writer 的恢复路径处理。终态市场、只读或非 owner 实例可用 SQL 构造本次响应，但不能安装、失效或复活全局记录。

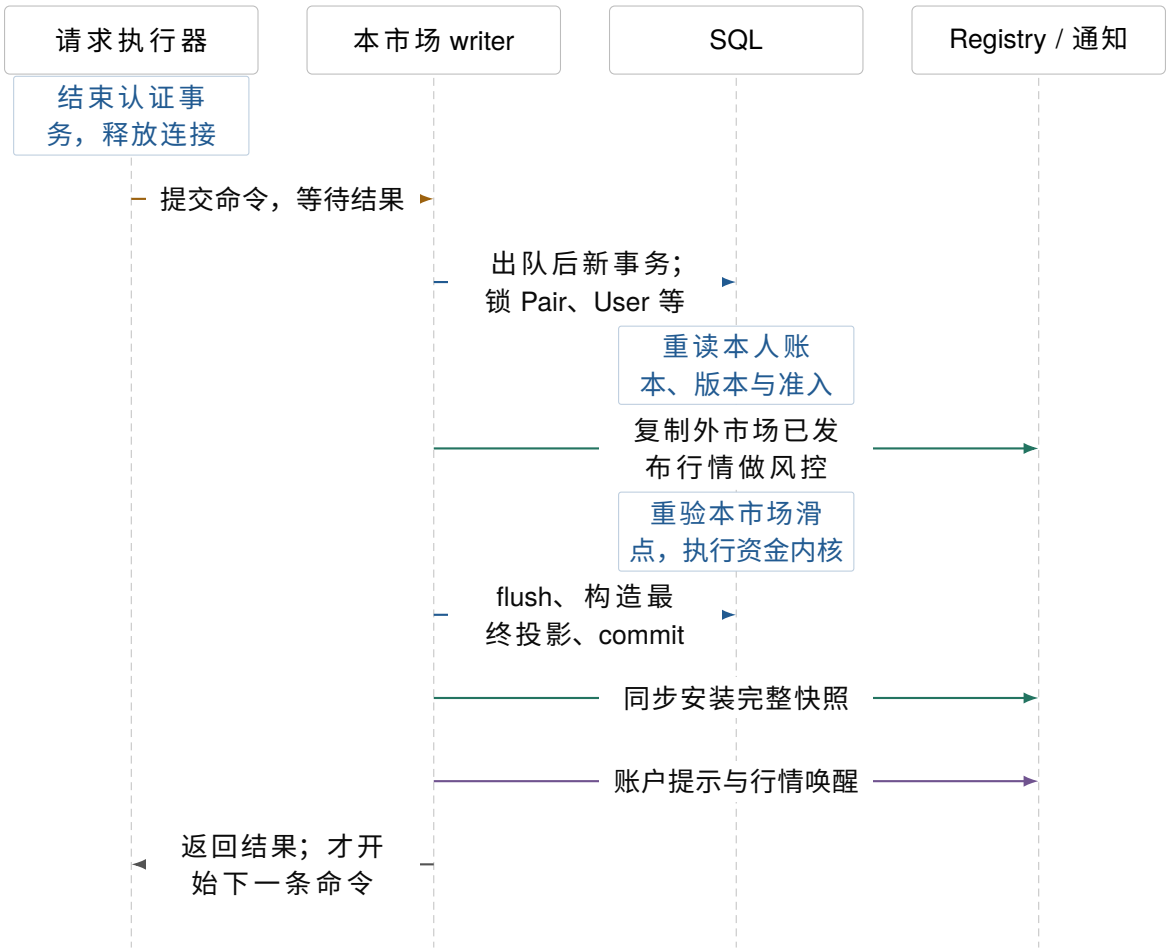
SQL 晚到也不能倒退状态。冷查结束时若已有健康的新记录，优先用它；若市场已删除或不可用，迟到的旧 SQL 不能反转这个状态。响应级冷读也不能提供 writer 风控所需的活跃发布状态。

> 报价只说明观察到的行情。下单时会按目标 writer 的真实执行状态重验价格和滑点；列表、详情、排行榜、成交量与历史也不因这次迁移全部变成零 SQL。

代码入口：`services/trading_prices.py`；`services/market_read_state.py`；`services/fx/trading.py`

3 FX 成交：一个 writer 负责完整提交顺序

请求侧只传 ID、金额等普通参数。认证和解析完成后，先结束数据库事务、释放连接和锁，再等待 submit；不能带着认证 session、ORM 对象或已锁账户排队。



首轮不改 FX 金融计算的权威。writer 调度现有事务内核，储备、资金、债务、成交回执、订单状态和审计仍在数据库事务中共同成立。内层 helper 不自行 commit，也不能回调另一个会 submit 的外层入口。

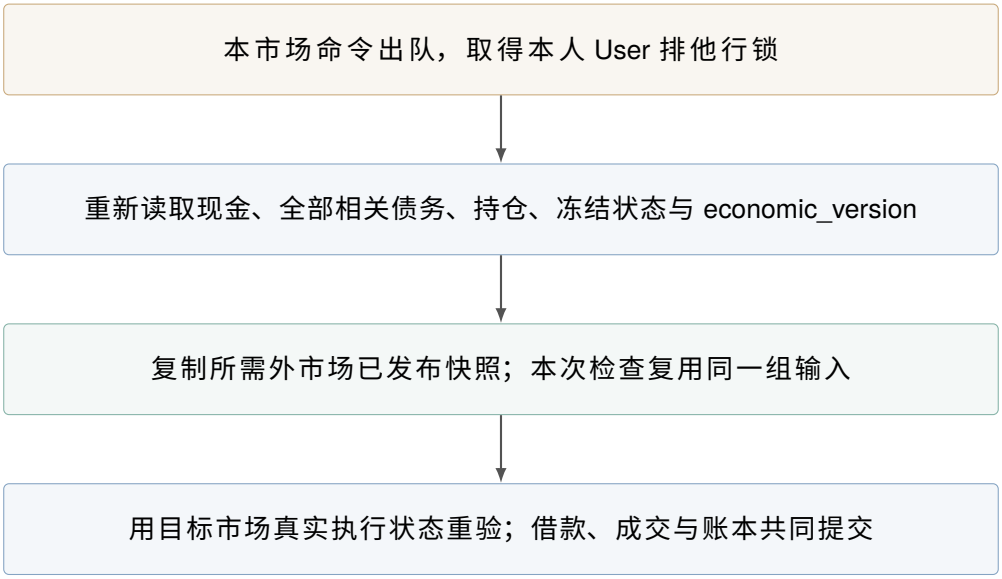
发布只有最终状态。flush 后按数据库实际精度构造快照；成功提交后同步安装，再发提示。复合交易各腿的中间值不会提前公开。提交、安装与通知完成后，consumer 才处理下一条命令。

队列顺序的含义。保证的是成功进入同一市场队列的次序，不是点击或网络到达时间。不同市场没有全局 FIFO；consumer 也不能持锁等待另一个市场 writer。

代码入口：`services/fx/writer.py`；`services/fx/writer_ops.py`；`services/fx/net_trading.py`

4 融资与跨市场：先锁本人，再用新账本检查

融资买入、跨零净交易都作为一个市场命令运行。借款和买入不能拆成“先借款提交，再排队成交”；各腿及执行回执仍在一笔资金事务中提交。



跨市场价格在什么时候取？固定为命令出队、取得账户锁后、资金变更前。只锁本人及本次实际写入的目标市场，不冻结其他抵押市场、不等它们的队列，也不向它们发送查询命令。不同快照可有不同观察时间；后续外市场变价不会撤回本次操作。

同一用户向两个市场各借 80。假设当前可承受新增负债 100，两条命令可以分别进入 A、B 队列。先取得 User 锁的一笔检查并提交；另一笔等待后，必须看到新增债务和持仓，再判断能否借 80。结果取决于第一笔后的真实组合风险，不能沿用旧的 100 额度，也不硬编码“第二笔必失败”。

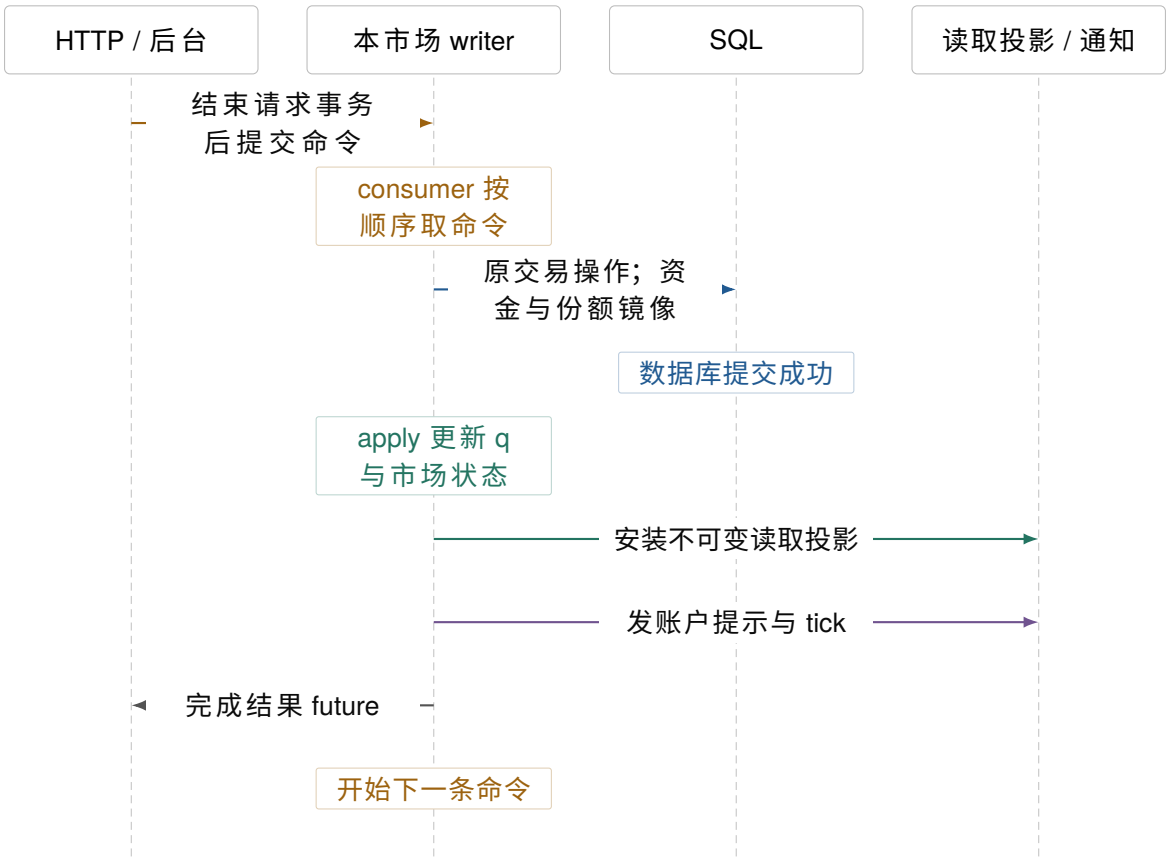
账户锁保护检查到提交。入队前发现的依赖和额度仅作提示或预取；锁后须重新确认，新增或清空的跨市场持仓不能漏算。行情中的市场投影不能替代刚重读的本人债务。共享 User 锁已经承担账户串行要求，本轮不新增账户队列或重复请求准入机制。

独立借还款、兑换和不改 AMM 的现金管理保留各自账户事务，遵守同一 User 锁纪律，并复用执行时行情。开平空仍检查本金、利息、金库和回购成本；幂等回执命中时返回原结果，不重新扣钱或安装旧行情。

代码入口：`services/fx/borrow_buy.py`；`services/fx/net_trading.py`；`services/fx/shorts.py`；`services/credit/`

5 LMSR：保留内存 q，补齐生命周期和通知

LMSR 沿用每市场 writer 和 consumer。买卖、关盘、恢复、结算、强平与实盘成交在同一市场队列运行；迁移补齐已有市场注册、重载和通知顺序。



writer 的 `q_dec` 继续保存 Decimal，定价浮点向量由它派生，参数 `b` 沿用 float。Registry 保存不可变副本，读者拿不到可修改 writer 状态的列表。FX 改用 writer 调度，并不意味着把 FX 同时改为内存储备权威。

账户提示跟在安装后。旧 GATE 模型中，op 内通知可能早于 apply；目标顺序把通知材料交回 consumer，等提交、apply 和 install 完成再发出。普通账户读取因此不必等待市场 GATE 来覆盖这一窗口。

注册和重载不能越过成交。新 ID 创建提交、关闭 session 后 bootstrap，成功后接单；已有 ID 的注册和重载进入原 writer 队列。consumer 内恢复直接调用内部加载路径，不能向自己 submit 后等待。不可用时失效旧投影，完成恢复后再接经济写入，避免旧 q 覆盖新成交。

代码入口：`services/market_writer.py`；`services/writer_ops.py`；`services/account_events.py`

6 刷新账户：账户稳定，附带行情独立

交易页需要余额、债务和持仓，也需要价格来估值。目标协议分别保证账户读取稳定和行情记录完整，不再承诺两者来自同一个提交截面。



接口	保留的检查	行情读取
context	完整账户、规则、依赖及经济版本稳定性	复制独立已发布记录，不排市场队列
versions	账户版本、规则版本与依赖	不为取得 stamp 等待市场写入
prices	在数据库核对请求市场属于用户依赖	授权后直接读完整快照

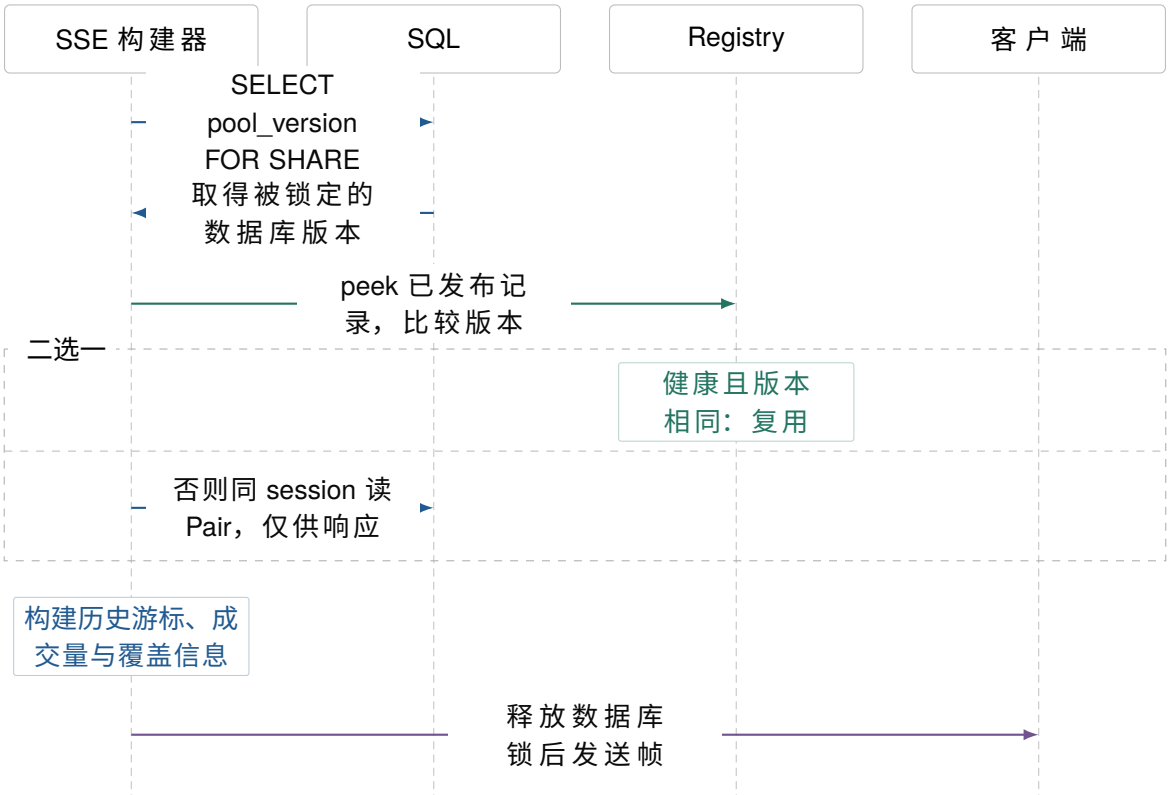
`minimum_group_stamps` 仅表示本响应附带价格的版本下限，不能再解释成“这一账户变更已被行情因果覆盖”。客户端分别合并 `economic_version` 和行情 stamp，不能互相替代。

行情组内 `as_of` 描述所附快照；context 顶层的 `as_of` 与 `server_time` 使用响应时钟。账户已更新而价格仍是上一份完整记录，在短暂发布窗口内属于已确认的取舍；风险执行仍按第5页的账户锁与真实账本检查。

代码入口：`services/credit/trading_context.py`；`services/trading_prices.py`；`schemas/trading_context.py`

7 公共 SSE：精确历史仍可能等数据库

普通报价与精确历史组合包的承诺不同。公共 FX SSE 若同时携带成交游标或历史前缀，继续证明行情与所宣称的历史覆盖相符，不能把旧报价和新历史拼成同一截面。



这条路径不经过通用 GATE。数据库 `FOR SHARE` 仍可能等待进行中的写入，也可能短暂阻挡写者；普通行情、账户估值和私人 SSE 初始价格不调用这条历史组合路径。

首帧保留订阅与 anchor 顺序。先订阅，再建立 anchor、读取快照和历史覆盖，行锁外发送。只读连接无法取得共享锁时，沿用 rollback 后读取并标记 `history_ready=false` 的降级语义。LMSR 的 ring、tail、封存、高水位和历史去重约束同样保留。

广播继续按市场复用。后台构建公共帧，已有 broker 分发编码 bytes；无订阅者时跳过构帧。历史 SQL 和精确滚动 24 小时成交量仍保留。

> 本轮取消普通读对市场队列的等待，不重做精确历史协议。因此不能声称所有 SSE 包都绝不等待数据库；把历史前缀也一起纳入 writer 发布，需要另行确认。

代码入口：`services/fx/trading.py`；`services/fx/market_data.py`；`api/v1/fx_stream.py`

8 私人 SSE：提示账户变化，共享价格输入

私人交易 SSE 使用独立短认证，鉴权连接不会陪长连接一直占用。账户提示和市场价格各有版本，不构成一份跨对象原子资金快照。



连上时。先订阅账户事件，再从短 session 重读依赖、规则与账户版本，随后复制初始已发布价格，形成 sync 首帧。首帧沿用账户版本稳定检查，价格不排市场队列。

连着时。PricingPublisher 共享市场输入，只在输入变化时投递价格组。账户提示会触发依赖复核，客户端再按版本补读；约每 15 秒继续复核准入、规则和依赖，长连接不免除后续授权检查。

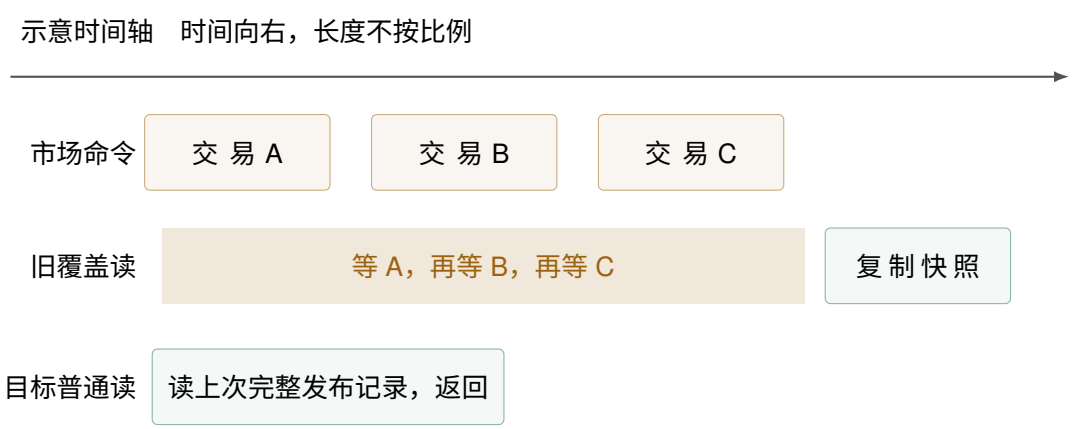
提示不替代持久事实。账户提示没有带回完整余额；行情唤醒只促使 publisher 检查，定时检查可弥补最终唤醒丢失。成交成功仍以数据库提交和持久回执为准，不能仅凭收到 SSE 就认定资金已经完整刷新。

> 提交后再提示可以减少无谓刷新，但不提供跨账户、跨市场原子一致性。私人 SSE 和普通账户估值都使用独立已提交行情；公共精确历史组合包保留第8页的例外。

代码入口：`api/v1/user_trading.py`；`services/trading_prices.py`；`services/account_events.py`

9　旧基线暴露了什么，新方案还要验证什么

旧的覆盖读取会申请共享 GATE，并让已有冲突写者先通过。读者可能等完一批成交，最后只用很短时间复制内存。新目标让普通读取直接返回最近完整发布记录。



旧轮 A/B：100 用户 / 200 SSE	旧 SQL 路径	GATE 覆盖读
读多写少：行情 Pair 输入 SELECT	1367	0
同阶段 SQL 总数	8925	7570
同阶段应用 CPU（一核为 100%）	39.75%	36.78%
stress：交易后 context P99	384ms	7135ms
stress：context GATE 等待 P99	未走此 GATE	6920ms
stress：context 持有 GATE 工作 P99	未走此 GATE	0.045ms

表内全部是旧报告数据，不是本轮唯一 writer 的结果。不同分布的 P99 不能相加当成单请求拆账；两版 stress 都因原 P99 保护提前停止，实际工作量有差异，不能据此推出交易容量变化的因果。

本轮最终对照待按计划完成：固定 `330f4e6` 与最终提交、同机同数据同请求密度，分别报告成功吞吐、错误率、行情/context P95/P99、队列等待、User/DB 锁等待、SQL 与 CPU。取消和失败请求不计入成功吞吐。普通读解除排队，也不会消除单市场串行、数据库提交或同账户争用。

代码入口：`output/fx-market-memory-ab-20261018/report.md`；
`docs/superpowers/plans/2026-10-18-single-writer-market-migration.md`

10 后台入口、生命周期与失败边界

入口	市场队列边界	事务中继续保证什么
条件单	创建、取消、触发成交及失败终态进入同 pair 队列；scheduler 只发现候选	出队后锁订单重验；成交资金与订单终态共同提交，不持订单锁等待 submit
强平、注撤资	发现事务结束后 submit；handler 锁内重新准备与检查	资金、幂等记录、审计及历史事件仍在各自原子事务中
市场管理	费率、储备、状态、归档、删除、空仓核销进入相应队列	保留容量与资金锁；调用者不持 capacity 锁等待队列
PvE	实盘和强制平仓复用交易命令；observation-only 也入同 pair 队列	profile 的 before_commit/on_fill 更新留在成交事务；observation 与成交仍是两次事务

PvE 账户生成、充值、启停和退休等不改 AMM 的管理继续独立事务，保留 Pair/User/Profile 锁及状态重验；不承诺和市场命令同队列顺序。observation 与成交之间不持锁，成交再次检查 profile/episode，内层不回调 submit wrapper。

创建、重载与删除。新市场先完成创建事务、关闭 session，再 bootstrap；已有市场重载排入原队列。删除后停止接该 ID 的新经济命令，已排队命令明确失败并完成 future，不能用 register 复活旧 ID。跨市场批处理逐市场提交，不嵌套等待其他 actor；离线 reset/修复仍要求 backend 停机。

失败发生在哪一侧

1. **提交前拒绝或回滚：**不公开未提交快照，旧的已提交记录仍有效。
2. **成功提交：**同步安装最终快照，随后提示并完成结果，再取下一条命令。
3. **提交成功但安装失败：**成交不能靠 rollback 撤销。标记不可用，沿现有 DB 重载路径恢复，不能用过期执行状态继续处理下一笔。
4. **等待超时或取消：**未开始的已取消命令可跳过；已开始的可能提交，客户端 timeout 不代表回滚，须依靠持久回执核对。

代码入口：`services/fx/orders.py`; `services/credit/execution.py`; `services/pve_fx/`; `api/v1/admin_fx.py`

11 对照代码与确认迁移进度

本文的图和时序描述批准后的目标。迁移可分阶段保留过渡 GATE，不能只凭搜索不到 GATE 引用就宣称资金正确；实际完成状态由总体验收统一记录。

要核对的问题	模块索引（相对 `backend/app/`）
FX 谁排序和提交	`services/fx/writer.py`、`services/fx/writer_ops.py`；交易内核见 `fx/trading.py`、`net_trading.py`、`borrow_buy.py`、`shorts.py`
LMSR 谁推进与发布	`services/market_writer.py`、`services/writer_ops.py`
谁保存不可变行情	`services/market_read_state.py`、`services/trading_prices.py`
账户与风控如何读取	`services/credit/`；账户写入口还须核对 User 锁与新账本读取
条件单、注撤资与 PvE	`services/fx/orders.py`、`liquidity.py`、`services/pve_fx/`
公共历史、私人 SSE	`services/fx/market_data.py`、`api/v1/fx_stream.py`、`api/v1/user_trading.py`
协议与前端版本合并	`schemas/trading_context.py`；前端 `src/stores/tradingSession.ts`

批准依据与进度。仓库文件

`docs/superpowers/plans/`

`2026-10-18-single-writer-market-migration.md`

固定执行检查时间点、历史协议、User 行锁方案及各入口边界；实施证据见该计划对应的 SDD ledger。本文不替代调用图审计、资金检查、PostgreSQL 并发检查或最终同口径性能对照。

本轮保留的边界。单经济写实例及 OWNERSHIP 不变；无跨进程报价同步。Registry 重启后重新加载，终态/只读的 SQL fallback 仅服务响应。不新增全内存账本、入队授信预留、账户队列、跨市场原子事务或历史发布协议。

历史数据来源。第10页数字来自

`output/fx-market-memory-ab-20261018/report.md`

及对应 GATE 统计；旧轮验证范围见同目录 `validation.md`。这些证据只说明旧基线，不证明当前目标已经实现或提速。

构建。在 `docs/latex` 直接运行两次

```
xelatex -interaction=nonstopmode -halt-on-error
trading-system-guide.tex
```

使用现有 XeLaTeX、中文字体与 TikZ；所有插图都是矢量图。